

ICODA

Handbook

Interactive Code Development and Analysis

ICODA (Interactive Code Development and Analysis) is a desktop application for developing software in small, reviewable steps. It combines a formal project specification, a derived architecture model, command-line LLM providers, isolated Git worktrees, build and test gates, and an explicit developer approval loop.

This handbook describes the behavior implemented in ICODA 0.1.0. `EVOLUTION.md` and `ICODA_PLAN.md` describe the design and its history; this file is the practical guide for using and maintaining the current application.

The screenshots are real Tk captures produced from deterministic acceptance fixtures. They show both C++ and Python projects so that the workflow is not mistaken for a language-specific design.

USER + MAINTAINER EDITION

VERSION 0.1.0 / 32 GUI SCREENSHOTS

Contents

1. The ICODA mental model	3
2. Installation and launch	4
3. Starting a project	5
4. Writing the specification	7
5. Main window tour	16
6. LLM selection	28
7. Architecture workflow	30
8. Implementation workflow	39
9. Build and test gates	46
10. Git, persistence, and recovery	48
11. Troubleshooting	50
12. Maintainer guide	53
13. Current limitations	56
14. Command reference	57
15. Glossary	58

Select a chapter in this contents page or use the PDF bookmarks to navigate. Each numbered figure link opens a full-page landscape screenshot after its chapter; the caption links back to the explanation. Screenshots retain their original resolution.

1. The ICODA mental model

ICODA is built around five ideas:

1. **The specification and source code are the project truths.** The specification says what the system should do and the source says what it currently does.
2. **The architecture view is derived.** ICODA parses the project and computes files, entities, calls, type relations, clusters, requirements, test evidence, and implementation status. The diagrams are not a separate model that must be maintained by hand.
3. **Development proceeds in phases.** A project moves from specification to architecture and then to implementation. The current phase controls which actions are available.
4. **Every proposed code change is isolated and verified.** The LLM response is applied in `.icoda/worktree`, built, tested, parsed, and compared with the current project before approval is enabled.
5. **The developer owns the decisions.** Architecture, implementation approaches, API signature changes, and failed gates remain visible decisions. Automatic approval is deliberately bounded.

ICODA currently supports C++ projects built with CMake presets and Python projects analysed with the standard library `ast` parser. The GUI is a native Tk desktop application; there is no web interface or background service.

2. Installation and launch

Common prerequisites

- Git
- Python 3.10 or newer with Tkinter
- At least one supported provider CLI if code generation will be used

For C++ projects, also install:

- CMake 3.28 or newer
- Ninja
- Clang and a compatible libclang
- clang-scan-deps when C++20 modules are used
- vcpkg when the project or its Code Profile requires it

On macOS, generated C++ module projects normally need Homebrew LLVM because Apple Clang does not provide the CMake module-scanning configuration used by the generated skeleton:

```
brew install llvm cmake ninja
```

Start ICODA

On macOS or Linux:

```
./icoda.bash  
./icoda.bash /absolute/path/to/project
```

On Windows:

```
icoda.cmd  
icoda.cmd C:\path\to\project
```

The launcher selects a suitable Python interpreter, checks the main prerequisites, creates `.icoda-venv`, installs ICODA in editable mode, and starts `icoda.py`. Warnings about optional C++ tools do not prevent Python-only work.

For contributor tools, install the development dependencies once:

```
.icoda-venv/bin/python -m pip install -e '.[dev]'
```

Use `.icoda-venv\Scripts\python.exe` instead on Windows.

Authenticate a provider

ICODA invokes provider command-line tools; it does not implement their login flows. Authenticate the selected CLI before starting a proposal. The currently enabled providers use:

```
claude          # complete the interactive login  
codex login
```

Provider availability and invocation compatibility can be checked without consuming model usage:

```
.icoda-venv/bin/python -m icoda_core.provider_check \  
--output .icoda-test-artifacts/provider-qualification.json
```

3. Starting a project

Create a new project

1. Choose **File > New Project...**
2. Select an empty directory.
3. Complete the Specification window.
4. Press **Validate**, resolve any reported problems, and press **Save**.
5. Build the generated skeleton as instructed by the confirmation dialog.
6. Return to ICODEA and choose **File > Reload**.

Saving the first specification creates only files that do not already exist. It generates either a C++ CMake skeleton or a Python skeleton according to the Code Profile, then moves the project into the architecture phase.

An empty directory initially receives the C++ defaults. To create a Python project, select Python and review all language-dependent Code Profile fields before saving. A practical Python profile is:

Field	Typical value
Standard	3.12
Test framework	pytest
Test runner	python -m pytest
Test files	tests/test_<module>.py
Source extension	.py
Module naming	snake_case
Class naming	PascalCase
Function naming	snake_case
Libraries	PyPI dependencies declared in pyproject.toml

Open an existing CMake project

1. Commit or otherwise safeguard the current project first.
2. Build it once with compile-command export enabled.
3. Choose **File > Open Project...** and select the project root.
4. If analysis reports no compile database, generate `compile_commands.json` as described below and reload.
5. Open **Project > Specification...** to add or edit the formal specification.

ICODA searches for the newest `compile_commands.json` at the project root, in `build/`, or in a direct child of `build/`. For a CMake project, use:

```
cmake --preset debug
cmake --build --preset debug
```

The preset must set `CMAKE_EXPORT_COMPILE_COMMANDS=ON`. The generated ICODEA presets already do this.

An existing CMake project with no ICODEA state starts in the `implementation` phase. Its existing functions are treated as implemented rather than as a new architecture skeleton.

Open an existing Python project

ICODA recursively analyses `.py` files without importing or executing them. A directory without `CMakeLists.txt` starts in the `specification` phase when it has no existing `.icoda/state.json`. Saving its first specification may add missing Python skeleton files. Commit the project first and inspect the resulting files, especially when the directory is not empty.

What happens on the first proposal

Before asking a provider, ICODEA prepares step 0:

- It creates `.icoda/` and the ignored proposal worktree.
- It initializes a Git repository if the project is not already its own repository.
- It builds and analyses the project when a fresh derived model is unavailable.
- It records the current skeleton/model as step 0.
- It creates an `icoda step 0: skeleton` commit.

Start with a clean, intentional working tree. On a project with no earlier ICODEA step, step 0 commits the files that constitute the starting point. Later uncommitted edits are refused until they are handled explicitly.

4. Writing the specification

Open the editor with **Project > Specification....** It has six pages and uses JSON Schema validation before save. Hover over a field or its label for an example.

Overview

- **Title:** a short project name; this is the only field that may not be empty.
- **Description:** what the program is, who uses it, and the outcome it provides.

Figure 01. Overview anchors the rest of the specification in a concrete product and audience.

Scope

- **Goals:** desired outcomes, one per line.
- **Not in scope:** features or qualities deliberately excluded.
- **Not allowed:** forbidden libraries, techniques, dependencies, or implementation choices.
- **Done when:** observable completion conditions.

Write exclusions as carefully as goals. They prevent a provider from broadening a bounded step into an attractive but unwanted subsystem.

Figure 02. Scope records what success includes, what it excludes, and which implementation shortcuts are forbidden.

Use cases

Use cases describe externally meaningful behavior. **Add** assigns stable IDs such as UC-1; **New** clears the form; **Remove** deletes the selected record. Include the main path and important alternate paths in the details.

For a complex system, use separate records for:

- normal operation;
- invalid input and boundary values;
- resource exhaustion and partial failure;
- cancellation, restart, and recovery;
- persistence and compatibility behavior;
- concurrency, timing, or ordering behavior where relevant.

Figure 03. Use cases are stable, numbered behavioral records rather than a single unstructured prompt.

Requirements

Each requirement receives an R-n ID, a priority, optional use-case references, and details. A requirement should be independently testable. Prefer measurable language such as:

R-4: Cancelling a queued job prevents it from starting. Cancellation completes within 100 ms and does not affect jobs that are already running.

Priorities mean:

- must: the project is unacceptable without it;
- should: important, but not a release blocker in every context;
- could: explicitly optional.

Use-case references must name existing UC - n IDs. Validation rejects unknown references and duplicate IDs.

Figure 04. Requirements connect measurable behavior to use cases and a release priority.

Decisions

Decisions record choices that should not be reopened in every prompt. Examples include the language, rendering or storage library, threading policy, public API constraints, ownership model, persistence format, and compatibility strategy. Record the rationale when it will help a later proposal preserve the intent rather than only the syntax.

Figure 05. Decisions preserve architectural intent so later proposals do not repeatedly reopen settled questions.

Code Profile

The Code Profile constrains generated code and supplies build/test conventions. Review at least:

- language and standard;
- C++ module use;
- target platforms;
- test framework and test runner;
- test-file and source-file conventions;
- module, class, and function naming;
- library policy;
- function-size limits;
- project-specific style rules.

Figure 06. The Code Profile turns project conventions into explicit generation and verification constraints.

Some profile fields, including the hard function limit and class-size limits, are retained in the JSON but are not currently exposed by the editor. Advanced changes can be made in `.icoda/specification.json` while ICODA is closed, then checked by reopening the editor and pressing **Validate**.

A quality checklist

Before leaving the specification phase, confirm that:

- every goal describes an outcome rather than an implementation activity;
- exclusions and forbidden choices are explicit;
- use cases cover normal, boundary, failure, and recovery paths;
- requirements are testable and include relevant limits;
- high-risk behavior has an independent oracle or reference result;
- important architecture choices and their rationale are recorded;

- completion requires passing tests, not merely compiling;
- the Code Profile matches the actual toolchain and project layout.

The current editor validates structure and references. It does not yet conduct an AI-guided interview, discover missing requirements, or prove that the specification is complete. That review remains a developer responsibility.

Link specification items to code

The Coverage view maps exact `@satisfies` tags to requirements. Requirements use their R-n ID; goals use their position as G-1, G-2, and so on.

C++ example:

```
/// @brief Cancels a job that has not started.  
/// @satisfies R-4  
bool cancel(JobId id);
```

Python example:

```
def cancel(job_id: JobId) -> bool:  
    """Cancel a queued job.  
  
    @satisfies R-4  
    """
```

Matching is exact; text similarity is not used. Because goal IDs are positional, avoid reordering goals casually after code has been tagged.

OverviewScopeUse casesRequirementsDecisionsCode profile

Title

Deterministic Task Engine

Description

A cross-platform library that schedules dependency-aware jobs for interactive and batch applications. Developers need predictable execution, cancellation, diagnostics, and reproducible tests under concurrency and resource pressure.

Validate

Save

Close

Figure 01. Overview anchors the rest of the specification in a concrete product and audience. [Back to explanation](#)

Overview

Scope

Use cases

Requirements

Decisions

Code profile

Goals	Execute dependency graphs without deadlock Keep scheduling overhead measurable and bounded Make failures and cancellation observable Produce deterministic evidence for every release gate
Not in scope	Distributed execution across machines A graphical workflow editor Persistent cloud queues
Not allowed	Detached worker threads Silent exception suppression Timing-only tests without a deterministic oracle
Done when	Every must requirement has traceable implementation and tests Reference schedules match for randomized dependency graphs Sanitizer, stress, recovery, and platform test suites pass

Validate

Save

Close

Figure 02. Scope records what success includes, what it excludes, and which implementation shortcuts are forbidden. [Back to explanation](#)

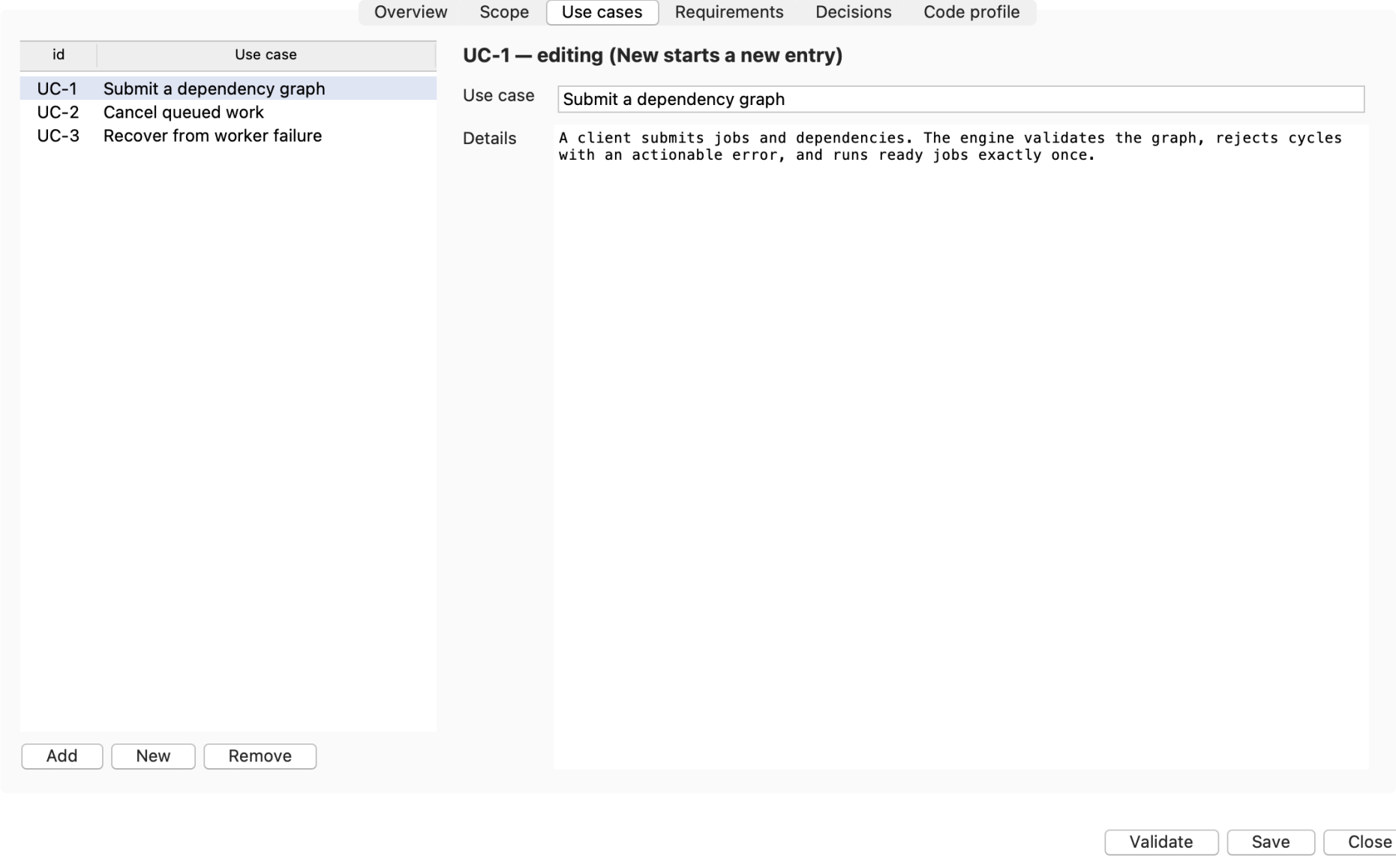


Figure 03. Use cases are stable, numbered behavioral records rather than a single unstructured prompt. [Back to explanation](#)

Overview

Scope

Use cases

Requirements

Decisions

Code profile

id	Requirement
R-1	Execute every ready job exactly once
R-2	Cancellation has bounded latency
R-3	Failures preserve diagnostic evidence
R-4	Sustain the target workload

Add

New

Remove

R-1 — editing (New starts a new entry)

Requirement

Execute every ready job exactly once

Priority

must

Use cases

UC-1

Details

For every acyclic graph, the observed execution set equals the reference topological schedule and every dependency completes before its dependant starts.

Validate

Save

Close

Figure 04. Requirements connect measurable behavior to use cases and a release priority. [Back to explanation](#)

OverviewScopeUse casesRequirementsDecisionsCode profile

id	Decision
D-1	Use a bounded work-stealing pool
D-2	Use deterministic model-based tests
D-3	Keep platform APIs behind adapters

AddNewRemove

D-1 — editing (New starts a new entry)

Decision

Use a bounded work-stealing pool

Why

It supports nested parallel work while keeping worker count and resource ownership explicit.

Validate

Save

Close

Figure 05. Decisions preserve architectural intent so later proposals do not repeatedly reopen settled questions. [Back to explanation](#)

Overview Scope Use cases Requirements Decisions Code profile

Language	C++
Standard	23
C++20 modules	☒
Platforms	☒ macOS ☒ Linux ☒ Windows
Test framework	doctest
Test runner	ctest --preset debug --output-on-failure
Test files	tests/<subsystem>_tests.cpp
Source extension	.cppm
Module naming	lower_snake_case
Class naming	PascalCase
Function naming	snake_case
Libraries	vcpkg manifest; single-header libraries vendored under third_party/
Max function lines	30
Style rules	Prefer STL algorithms to hand-written loops; lambdas where they make sense; templates for reuse. Every entity carries a Doxygen comment with @brief and, where a requirement applies, @satisfies. Platform independence: no platform API without a portable wrapper.

Validate Save Close

Figure 06. The Code Profile turns project conventions into explicit generation and verification constraints. [Back to explanation](#)

5. Main window tour

Menus

File creates, opens, reloads, and remembers projects. **Project** opens the specification and libclang chooser, starts proposals, approves the architecture, undoes the last approved step, or commits manual edits. **View** opens the analysis surfaces, fits diagrams, and toggles coverage colours.

Shared graph controls

The controls above the view tabs affect the reusable diagrams:

- **Filter** hides nodes that do not match.
- **Neighborhood** dims nodes more than the chosen graph distance from the selected entity; 0 disables it.
- **Collapse all** collapses the shared Project -> Cluster -> File -> Class -> Function hierarchy.
- The -, **Fit**, **100%**, and + controls adjust the current diagram.

The filter accepts space-separated terms. Plain text searches names; qualified terms include:

```
name:render
kind:function
status:stub
covered:true
stale:false
cluster:core
namespace:app
edge:calls
```

Terms can be combined. Quoted values are accepted. An invalid or unfinished quote is handled as plain text rather than crashing the filter.

Figure 07. Qualified filters narrow a large graph to the entities relevant to the current question.

Diagram interaction

- Scroll over a diagram to zoom around the pointer.
- Drag with the left or middle mouse button to pan.
- Click + or - in the hierarchy panel to expand or collapse a level.
- Hover over a node for source and status details.
- Double-click a source entity to open it in the configured editor.
- Right-click a node for actions that are valid in the current phase.

Figure 08. Neighborhood mode keeps local context prominent while retaining the shape of the surrounding graph.

The right-click menu can show an entity's introducing step, focus an architecture proposal, override the current implementation target, run recorded tests, pin or rename a cluster, or pin a file to a chosen cluster. Disabled items explain that the action is not valid for the current node or workflow state.

Figure 09. The hierarchy remains available beside every graph; cluster names and file pins survive reloads.

Views

File View shows project files, dependency clusters, typed relations, and external libraries. Clicking a file lists its entities in the right sidebar. Cluster names and pins are persisted.

Figure 10. File View joins dependency structure with the expandable source hierarchy.

Call View shows a function-level graph from a selected root. Use **From main**, **Depth**, and **callers** to change the traversal. Recursion is drawn as a loop; uncertain dynamic calls are dashed and marked ?. A proposal's added and changed entities receive distinct outlines.

Figure 11. Dashed edges and a question mark preserve uncertainty instead of presenting dynamic dispatch as a false fact.

Class View shows classes and structs with fields and methods. It distinguishes inheritance, composition, aggregation, and usage relations. Callable members include their implementation status.

Figure 12. Class View makes member signatures and implementation status reviewable as architecture.

Mind Map is a persistent hierarchy of clusters, files, classes, and functions. Nodes include status, requirement IDs, and the introducing step. Clicking an expandable node changes its persisted expansion state; clicking a node with history also loads that step into the review panel.

Figure 13. The Mind Map connects structure to requirements and the step that introduced it.

Coverage has two independent sections:

- specification coverage from exact `@satisfies` tags;
- callable test coverage from successful recorded test runs and call reachability.

Double-click a callable row to open its source location. Specification coverage and test coverage are intentionally different facts.

Figure 14. Requirement traceability and executed test evidence are reported as separate coverage dimensions.

Issues lists advisory code-rule findings such as missing documentation, missing `@satisfies`, large functions, large classes, too many parameters, direct platform API use, and missing successful test evidence. Double-click a row to open the source.

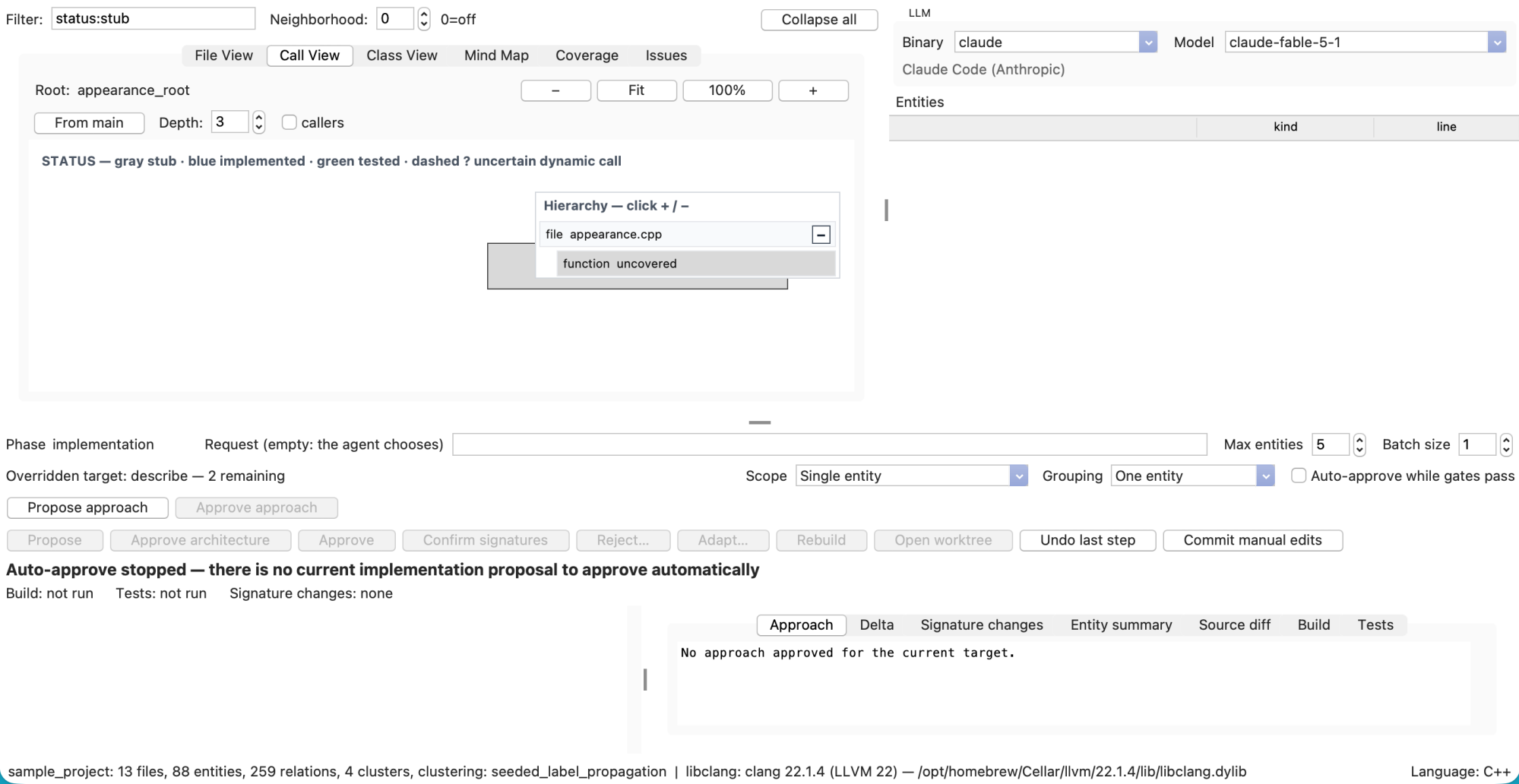
Figure 15. Issues turn Code Profile rules and missing evidence into a source-oriented review queue.

Status and colours

In normal status mode, callable nodes are gray for `stub`, blue for `implemented`, and green for `tested`. Coverage colour mode uses green for covered and red for uncovered. A red warning marker means the whole derived model or an entity's recorded test evidence is stale.

Issue findings are currently advisory except where a specific workflow gate, such as grouped test coverage, explicitly enforces them.

Figure 16. Coverage colour mode makes recorded test reachability visible directly on a graph.



Filter:

Neighborhood: 0=off

Collapse all

File ViewCall ViewClass ViewMind MapCoverageIssues

Root: appearance_root

From main

Depth:

☐ callers

STATUS — gray stub · blue implemented · green tested · dashed ? uncertain dynamic call

appearance_root

Hierarchy — click + / -

cluster Pinned Application Core

cluster third_party

cluster util

cluster tests

file appearance.cpp

file appearance_test.cpp

LLM

Binary

Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

Phase implementation

Request (empty: the agent chooses)

Max entities

Batch size

Overridden target: describe — 2 remaining

Scope

Grouping

☐ Auto-approve while gates pass

Propose approach

Approve approach

Propose

Approve architecture

Approve

Confirm signatures

Reject...

Adapt...

Rebuild

Open worktree

Undo last step

Commit manual edits

Auto-approve stopped — there is no current implementation proposal to approve automatically

Build: not run

Tests: not run

Signature changes: none

Approach

Delta

Signature changes

Entity summary

Source diff

Build

Tests

No approach approved for the current target.

sample_project: 13 files, 88 entities, 259 relations, 4 clusters, clustering: seeded_label_propagation | libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib

Language: C++

Figure 08. Neighborhood mode keeps local context prominent while retaining the shape of the surrounding graph. [Back to explanation](#)

Filter:

Neighborhood: 0=off

Collapse all

File View | Call View | Class View | Mind Map | Coverage | Issues

STATUS — gray stub · blue implemented · green tested

Hierarchy — click + / -

cluster Pinned Application Core

cluster third_party

cluster util

cluster tests

file config.cppm

file simulation.cppm

file geometry.cppm

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

Phase implementation

Request (empty: the agent chooses)

Max entities Batch size

Overridden target: describe — 2 remaining

Scope Grouping ☐ Auto-approve while gates pass

Propose approach

Approve approach

Propose

Approve architecture

Approve

Confirm signatures

Reject...

Adapt...

Rebuild

Open worktree

Undo last step

Commit manual edits

Auto-approve stopped — there is no current implementation proposal to approve automatically

Build: not run Tests: not run Signature changes: none

Approach | Delta | Signature changes | Entity summary | Source diff | Build | Tests

No approach approved for the current target.

automatic approval disabled

Language: C++

Figure 09. The hierarchy remains available beside every graph; cluster names and file pins survive reloads. [Back to explanation](#)

Filter:

Neighborhood: 0=off

Collapse all

File View | Call View | Class View | Mind Map | Coverage | Issues

STATUS — gray stub · blue implemented · green tested

Hierarchy — click + / -

cluster tests

file service.py

file test_service.py

function normalize

class Store

method load

function main

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

Phase specification

Request (empty: the agent chooses)

Max entities Batch size

Implementation queue: inactive

Scope Grouping ☐ Auto-approve while gates pass

Propose approach

Approve approach

Propose

Approve architecture

Approve

Confirm signatures

Reject...

Adapt...

Rebuild

Open worktree

Undo last step

Commit manual edits

No proposal

Build: not run Tests: not run Signature changes: none

Approach | Delta | Signature changes | Entity summary | Source diff | Build | Tests

python-project: 2 files, 5 entities, 7 relations, 1 clusters, clustering: seeded_label_propagation | Python ast

Language: Python

Figure 10. File View joins dependency structure with the expandable source hierarchy. [Back to explanation](#)

Version 0.1.0 | Illustrated edition

21

Filter:

Neighborhood: 0=off

Collapse all

File ViewCall ViewClass ViewMind MapCoverageIssues

Root: dispatch

From mainDepth: ☐ callers

STATUS — gray stub · blue implemented · green tested · dashed ? uncertain dynamic call

dispatch

fixed_member

base_virtual_member

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

Phase implementationRequest (empty: the agent chooses)

Max entities Batch size

Overridden target: describe — 2 remaining

Scope Grouping ☐ Auto-approve while gates pass

Propose approachApprove approach

ProposeApprove architectureApproveConfirm signaturesReject...Adapt...RebuildOpen worktreeUndo last stepCommit manual edits

Auto-approve stopped — there is no current implementation proposal to approve automatically

Build: not runTests: not runSignature changes: none

ApproachDeltaSignature changesEntity summarySource diffBuildTests

No approach approved for the current target.

sample_project: 13 files, 88 entities, 259 relations, 4 clusters, clustering: seeded_label_propagation | libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib

Language: C++

Figure 11. Dashed edges and a question mark preserve uncertainty instead of presenting dynamic dispatch as a false fact. [Back to explanation](#)

Filter:

Neighborhood: 0=off

Collapse all

File ViewCall ViewClass ViewMind MapCoverageIssues

1 class / structs

STATUS — gray stub · blue implemented · green tested

class service.Store

0 data · 1 methods

def load(self, value: str) -> str [implemented]

Hierarchy — click + / -

cluster tests

file service.py

file test_service.py

function normalize

class Store

method load

function main

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

Phase specification

Request (empty: the agent chooses)

Max entities Batch size

Implementation queue: inactive

Scope Grouping ☐ Auto-approve while gates pass

Propose approach

Approve approach

Propose

Approve architecture

Approve

Confirm signatures

Reject...

Adapt...

Rebuild

Open worktree

Undo last step

Commit manual edits

No proposal

Build: not run Tests: not run Signature changes: none

Approach

Delta

Signature changes

Entity summary

Source diff

Build

Tests

python-project: 2 files, 5 entities, 7 relations, 1 clusters, clustering: seeded_label_propagation | Python ast

Language: Python

Figure 12. Class View makes member signatures and implementation status reviewable as architecture. [Back to explanation](#)

Version 0.1.0 | Illustrated edition

23

Filter:

Neighborhood: 0=off

Collapse all

File ViewCall ViewClass ViewMind MapCoverageIssues

Mind map: 73 nodes · 9 visible

STATUS — gray stub · blue implemented · green tested

Cluster · src
implemented · R-8 · step unknown

File · main.cpp
implemented · R-8 · step unknown

Function · main
implemented · R-8 · step unknown

Cluster · app
implemented · R-7, R-8 · step unknown

Cluster · core
implemented · R-1, R-2, R-3 · step unknown

Cluster · render
implemented · R-4 · step unknown

Cluster · third_party
implemented · no requirements · step unknown

Cluster · util
implemented · R-5, R-6 · step unknown

Cluster · tests
unknown · no requirements · step unknown

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

Phase implementation

Request (empty: the agent chooses)

Max entities Batch size

Overridden target: describe — 2 remaining

Scope Grouping ☐ Auto-approve while gates pass

Propose approach

Approve approach

Propose

Approve architecture

Approve

Confirm signatures

Reject...

Adapt...

Rebuild

Open worktree

Undo last step

Commit manual edits

Auto-approve stopped — there is no current implementation proposal to approve automatically

Build: not run Tests: not run Signature changes: none

ApproachDeltaSignature changesEntity summarySource diffBuildTests

No approach approved for the current target.

sample_project: 13 files, 88 entities, 259 relations, 4 clusters, clustering: seeded_label_propagation | libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib

Language: C++

Figure 13. The Mind Map connects structure to requirements and the step that introduced it. [Back to explanation](#)

Version 0.1.0 | Illustrated edition

24

Filter:

Neighborhood: 0=off

Collapse all

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

File View

Call View

Class View

Mind Map

Coverage

Issues

Specification coverage: 2/3 goals and requirements covered · 1 UNCOVERED

Coverage requires an exact @satisfies identifier; goals use G-1, G-2,

Coverage	ID	Kind	Specification item	Implementing entities
Covered	G-1	Goal	Keep the developer informed	centroid
Covered	R-54	Requirement	Show implementing code	centroid
UNCOVERED	R-55	Requirement	Show newly requested work	No implementing entity

Test coverage: 2/44 callables covered · 42 uncovered

Coverage evidence comes from successful recorded test runs and call reachability.

Coverage	Callable	How covered: tests	Evidence
----------	----------	--------------------	----------

Phase implementation

Request (empty: the agent chooses)

Max entities Batch size

Overridden target: describe — 2 remaining

Scope Grouping ☐ Auto-approve while gates pass

Propose approach

Approve approach

Propose

Approve architecture

Approve

Confirm signatures

Reject...

Adapt...

Rebuild

Open worktree

Undo last step

Commit manual edits

Auto-approve stopped — there is no current implementation proposal to approve automatically

Build: not run Tests: not run Signature changes: none

Approach

Delta

Signature changes

Entity summary

Source diff

Build

Tests

No approach approved for the current target.

sample_project: 13 files, 88 entities, 259 relations, 4 clusters, clustering: seeded_label_propagation | libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib

Language: C++

Figure 14. Requirement traceability and executed test evidence are reported as separate coverage dimensions. [Back to explanation](#)

Filter:

Neighborhood: 0=off

Collapse all

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

File View Call View Class View Mind Map Coverage Issues

Rule checks: 159 issues · 2 errors · 157 warnings

Double-click an issue to open its offending entity in the editor.

Severity	Rule	Entity	What to do	Location
Error	function-lines	acceptance::open_nativ	acceptance::open_native is 57 lines; split it below	main.cpp:2
Error	platform-api	acceptance::open_nativ	acceptance::open_native calls platform API Creat	main.cpp:4
Warning	many-parameter	acceptance::open_nativ	acceptance::open_native has 6 parameters; grou	main.cpp:2
Warning	missing-doxyger	acceptance::open_nativ	acceptance::open_native has no Doxygen comm	main.cpp:2
Warning	missing-satisfies	acceptance::open_nativ	acceptance::open_native has no @satisfies tag; li	main.cpp:2
Warning	missing-test	acceptance::open_nativ	acceptance::open_native has no successful test	main.cpp:2
Warning	missing-satisfies	kDefaultWidth	kDefaultWidth has no @satisfies tag; link it to its	src/app/config.cp
Warning	missing-satisfies	kDefaultHeight	kDefaultHeight has no @satisfies tag; link it to its	src/app/config.cp
Warning	missing-doxyger	Config::width	Config::width has no Doxygen comment; docume	src/app/config.cp
Warning	missing-satisfies	Config::width	Config::width has no @satisfies tag; link it to its r	src/app/config.cp
Warning	missing-doxyger	Config::height	Config::height has no Doxygen comment; docum	src/app/config.cp

Phase implementation Request (empty: the agent chooses)

Max entities Batch size

Overridden target: describe — 2 remaining

Scope Grouping ☐ Auto-approve while gates pass

Propose approach Approve approach

Propose Approve architecture Approve Confirm signatures Reject... Adapt... Rebuild Open worktree Undo last step Commit manual edits

Auto-approve stopped — there is no current implementation proposal to approve automatically

Build: not run Tests: not run Signature changes: none

Approach Delta Signature changes Entity summary Source diff Build Tests

No approach approved for the current target.

sample_project: 13 files, 88 entities, 259 relations, 4 clusters, clustering: seeded_label_propagation | libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib

Language: C++

Figure 15. Issues turn Code Profile rules and missing evidence into a source-oriented review queue. [Back to explanation](#)

Filter:

Neighborhood: 0=off

Collapse all

File ViewCall ViewClass ViewMind MapCoverageIssues

Root: appearance_root

From main

Depth:

☐ callers

COVERAGE — green covered · red uncovered · dashed ? uncertain dynamic call

appearance_root

Hierarchy — click + / -

cluster Pinned Application Core

cluster third_party

cluster util

cluster tests

file appearance.cpp

file appearance_test.cpp

LLM

Binary

Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

Phase implementation

Request (empty: the agent chooses)

Max entities

Batch size

Overridden target: describe — 2 remaining

Scope

Grouping

☐ Auto-approve while gates pass

Propose approach

Approve approach

Propose

Approve architecture

Approve

Confirm signatures

Reject...

Adapt...

Rebuild

Open worktree

Undo last step

Commit manual edits

Auto-approve stopped — there is no current implementation proposal to approve automatically

Build: not run Tests: not run Signature changes: none

Approach

Delta

Signature changes

Entity summary

Source diff

Build

Tests

No approach approved for the current target.

sample_project: 13 files, 88 entities, 259 relations, 4 clusters, clustering: seeded_label_propagation | libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib

Language: C++

Figure 16. Coverage colour mode makes recorded test reachability visible directly on a graph. [Back to explanation](#)

6. LLM selection

The compact LLM panel contains two editable fields:

- **Binary** is the executable name or full path.
- **Model** contains the models configured for the resolved binary, but also accepts a typed model ID.

Figure 17. The selector keeps the executable, provider-specific model, and resolved provider identity together in one compact panel.

Changing the binary updates the model list and remembers the model used for each provider during the session. The selection is stored per project in ignored `.icoda/ui.json` and as the default in the user configuration.

The shipped provider registry contains Claude Code, Codex CLI, Gemini CLI, OpenCode, Aider, GitHub Copilot CLI, and Qwen Code. Only Claude and Codex are currently enabled and marked verified. The other entries, including Gemini, remain hidden from the normal Binary dropdown until their invocation is qualified locally and their `enabled/verified` fields in `icoda_core/providers.json` are deliberately updated.

A full path still resolves to a provider by executable basename. An unknown binary can be typed, but ICODA cannot invoke it because it has no command template; the field displays that warning directly.

Provider calls are bounded to 30 minutes. ICODA retries rate-limit responses up to three times, following a provider-supplied delay when it can parse one. A malformed proposal is also re-requested up to three times with validation or build feedback.

Filter:

Neighborhood: 0=off

Collapse all

File View

Call View

Class View

Mind Map

Coverage

Issues

-

Fit

100%

+

STATUS — gray stub · blue implemented · green tested

Hierarchy — click + / -

cluster src

cluster app

cluster core

cluster render

cluster third_party

cluster util

cluster tests

file confia.cnm

LLM

Binary

codex

Model

gpt-5.6-sol

Codex CLI (OpenAI)

Entities

	kind	line
--	------	------

Phase implementation

Request (empty: the agent chooses)

Max entities

Batch size

Current target: Renderer::draw — 3 remaining

Scope

Queue order

Grouping

One entity

☐ Auto-approve while gates pass

Propose approach

Approve approach

Propose

Approve architecture

Approve

Confirm signatures

Reject...

Adapt...

Rebuild

Open worktree

Undo last step

Commit manual edits

No proposal

Build: not run

Tests: not run

Signature changes: none

Approach

Delta

Signature changes

Entity summary

Source diff

Build

Tests

No approach approved for the current target.

sample_project: 13 files, 88 entities, 259 relations, 7 clusters, clustering: seeded_label_propagation | libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib

Language: C++

Figure 17. The selector keeps the executable, provider-specific model, and resolved provider identity together in one compact panel. [Back to explanation](#)

7. Architecture workflow

The architecture phase grows the structural skeleton in small increments. Functions introduced here should remain stubs; behavior is implemented later.

Propose an architecture step

1. Confirm the status bar shows a fresh model and the phase is architecture.
2. Select a provider binary and model.
3. Optionally enter a precise **Request**. If it is empty, the provider chooses the next step from the specification and current model.
4. Set **Max entities** to bound the architecture delta.
5. Press **Propose** or right-click a diagram node and choose **Propose the next step here**.

ICODA builds a prompt from the specification, Code Profile, derived model, build files, relevant rule issues, earlier rejection reasons, and optional focus. The provider returns a structured response. ICODEA applies it in the proposal worktree, builds it, runs tests, reparses it, and computes the architecture delta.

An architecture proposal is rejected automatically if it exceeds the entity budget, changes implementation bodies where only stubs are expected, fails to build, fails to test, cannot be parsed, or violates the response contract.

Figure 18. A live architecture proposal exposes both the structural change and its gate status before approval.

Review a proposal

The lower panel contains:

- the provider rationale and questions;
- **Delta**: added, removed, changed, and renamed entities;
- **Signature changes**: previous and proposed declarations;
- **Entity summary**: editable structured intent for adaptation;
- **Source diff**: the exact candidate patch;
- **Build** and **Tests**: captured gate output.

Review the source diff and the resulting diagrams, not only the rationale. Build and test success prove the configured checks passed; they do not prove the change meets the specification.

Figure 19. Delta summarizes the model-level effect rather than forcing reviewers to infer it from text.

Figure 20. Entity summary provides structured intent that can be edited and returned as hard adaptation constraints.

Figure 21. Source diff remains the authoritative view of what the proposal would actually change.

Decide

- **Approve** promotes the worktree changes, rebuilds and retests the real project, appends the step record, and creates one Git commit.
- **Reject...** records a reason. The next proposal receives that reason as feedback.

- **Adapt...** uses edits made in the Entity summary as hard constraints. If the summary is unchanged, ICODA asks for a semicolon-separated free-text instruction.
- **Open worktree** opens the proposal checkout for manual inspection or editing.
- **Rebuild** reruns build, tests, parsing, and delta computation after a manual worktree edit.

Figure 22. Rejection is a recorded decision; its reason becomes feedback for the next attempt.

If an existing entity signature changes, **Approve** remains disabled until **Confirm signatures** is pressed. Confirmation applies only to the proposal currently displayed and is not persisted as a general permission.

Figure 23. An API change receives its own gate even when build and tests already pass.

Finish architecture

When the skeleton is adequate and no live proposal is waiting for a decision, press **Approve architecture**. After confirmation, ICODA records the phase transition, constructs the implementation queue, reloads the project, and enables the implementation controls.

Figure 24. Approving architecture is an explicit phase gate, not an incidental result of approving one proposal.

Filter:

Neighborhood: 0=off

Collapse all

File View | Call View | Class View | Mind Map | Coverage | Issues

Root: service.Formatter.normalize

From main

Depth:

☐ callers

STATUS — gray stub · blue implemented · green tested · dashed ? uncertain dynamic call

Hierarchy — click + / -

cluster .

file project.py

service.Formatter.normalize

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

Phase architecture

Request (empty: the agent chooses)

Max entities Batch size

Implementation queue: inactive

Scope Grouping ☐ Auto-approve while gates pass

Propose approach

Approve approach

Propose

Approve architecture

Approve

Confirm signatures

Reject...

Adapt...

Rebuild

Open worktree

Undo last step

Commit manual edits

Step 1: Add formatter architecture (attempt 1)

Build: passed Tests: passed Signature changes: none

Keep this atomic and derive the review delta from the changed source.

Approach | Delta | Signature changes | Entity summary | Source diff | Build | Tests

3 entities added, 0 changed, 0 removed; files: service.py
+ class service.Formatter
+ method service.Formatter.normalize def normalize(self, value: str) -> str

step 1: proposal ready — approve, reject or adapt

Language: C++

Figure 18. A live architecture proposal exposes both the structural change and its gate status before approval. [Back to explanation](#)

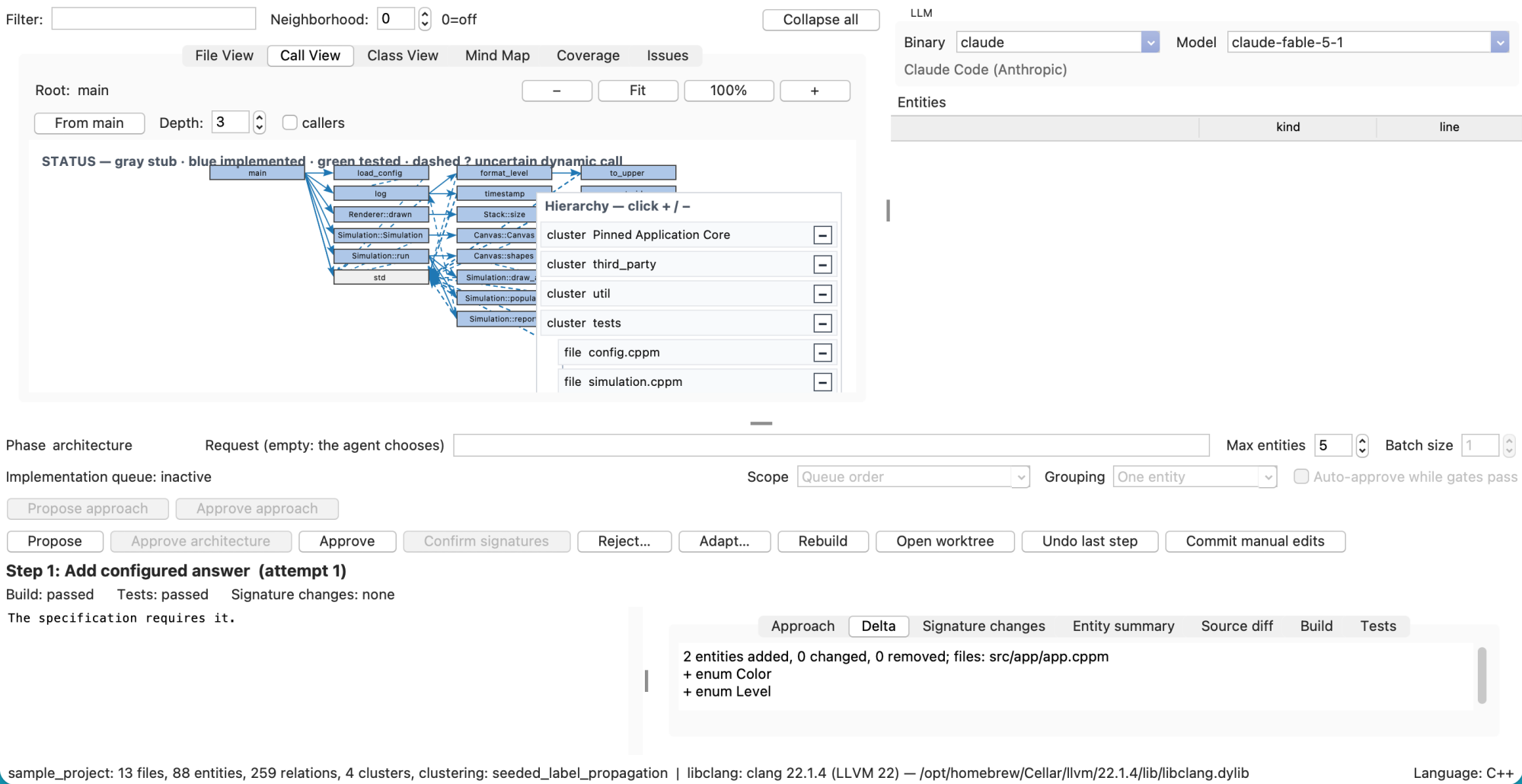


Figure 19. Delta summarizes the model-level effect rather than forcing reviewers to infer it from text. [Back to explanation](#)

Filter:

Neighborhood: 0=off

Collapse all

File ViewCall ViewClass ViewMind MapCoverageIssues

Root: main

From mainDepth: ☐ callers

STATUS — gray stub · blue implemented · green tested · dashed ? uncertain dynamic call

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

Phase architecture

Request (empty: the agent chooses)

Max entities Batch size

Implementation queue: inactive

Scope Grouping ☐ Auto-approve while gates pass

Propose approachApprove approach

ProposeApprove architectureApproveConfirm signaturesReject...Adapt...RebuildOpen worktreeUndo last stepCommit manual edits

Step 1: Add configured answer (attempt 1)

Build: passed Tests: passed Signature changes: none

The specification requires it.

ApproachDeltaSignature changesEntity summarySource diffBuildTests

```
[
{
  "name": "app::answer",
  ...
}
```

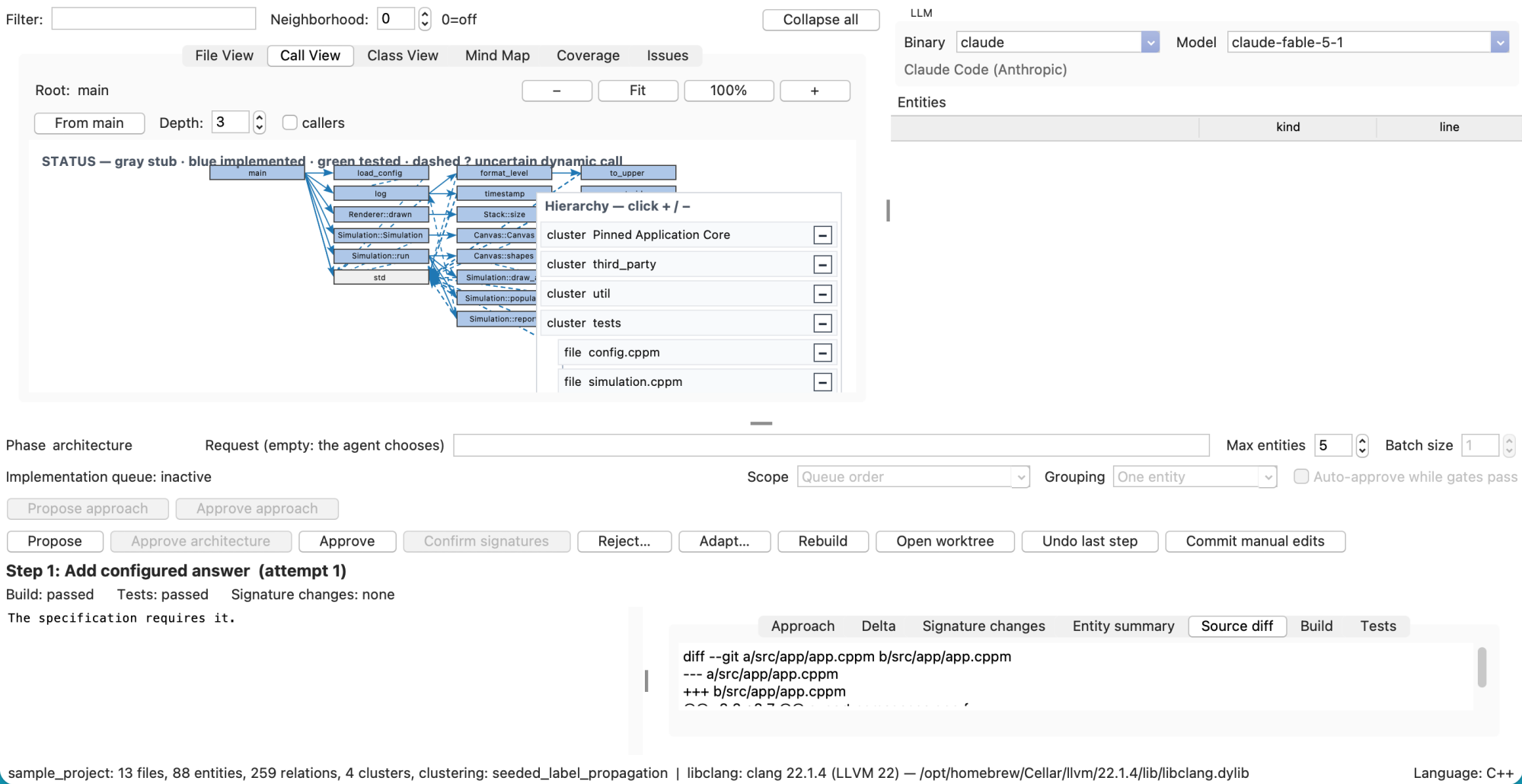
sample_project: 13 files, 88 entities, 259 relations, 4 clusters, clustering: seeded_label_propagation | libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib

Language: C++

Figure 20. Entity summary provides structured intent that can be edited and returned as hard adaptation constraints. [Back to explanation](#)

Version 0.1.0 | Illustrated edition

34



Filter:

Neighborhood: 0=off

Collapse all

File ViewCall ViewClass ViewMind MapCoverageIssues

Root: service.Formatter.normalize

From main

Depth:

☐ callers

STATUS — gray stub · blue implemented · green tested · dashed ? uncertain dynamic call

Hierarchy — click + / -

cluster .

file project.py

service.Formatter.normalize

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

Phase architecture

Request (empty: the agent chooses)

Max entities Batch size

Implementation queue: inactive

Scope Grouping ☐ Auto-approve while gates pass

Propose approach

Approve approach

Propose

Approve architecture

Approve

Confirm signatures

Reject...

Adapt...

Rebuild

Open worktree

Undo last step

Commit manual edits

Step 1: Draft formatter architecture (attempt 1)

Build: passed Tests: passed Signature changes: none

Keep this atomic and derive the review delta from the changed source.

ApproachDeltaSignature changesEntity summarySource diffBuildTests

diff --git a/service.py b/service.py
new file mode 100644
--- /dev/null
.....

step 1: proposal ready — approve, reject or adapt

Language: C++

Figure 22. Rejection is a recorded decision; its reason becomes feedback for the next attempt. [Back to explanation](#)

Filter:

Neighborhood: 0=off

Collapse all

File ViewCall ViewClass ViewMind MapCoverageIssues

Root: main

From mainDepth: ☐ callers

STATUS — gray stub · blue implemented · green tested · dashed ? uncertain dynamic call

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

Phase architectureRequest (empty: the agent chooses)

Implementation queue: inactive

Scope Grouping ☐ Auto-approve while gates pass

Propose approachApprove approach

ProposeApprove architectureApproveConfirm signaturesReject...Adapt...RebuildOpen worktreeUndo last stepCommit manual edits

Step 2: Change existing signature (attempt 1)

Build: passedTests: passedSignature changes: 1 — confirmation required

The API needs it.

ApproachDeltaSignature changesEntity summarySource diffBuildTests

centroid

Previous: Point centroid(const std::vector<Point> &)

Proposed: Point centroid(const std::vector<Point> &) /* confirmed change */

step 2: proposal ready — approve, reject or adapt

Language: C++

Figure 23. An API change receives its own gate even when build and tests already pass. [Back to explanation](#)

Filter:

Neighborhood: 0=off

Collapse all

File ViewCall ViewClass ViewMind MapCoverageIssues

STATUS — gray stub · blue implemented · green tested

Hierarchy — click + / -

cluster .

file project.py

file service.py

class Formatter

method normalize

function main

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

Phase architecture

Request (empty: the agent chooses)

Max entities Batch size

Implementation queue: inactive

Scope Grouping ☐ Auto-approve while gates pass

Propose approach

Approve approach

Propose

Approve architecture

Approve

Confirm signatures

Reject...

Adapt...

Rebuild

Open worktree

Undo last step

Commit manual edits

No proposal

Build: not run Tests: not run Signature changes: none

Approach

Delta

Signature changes

Entity summary

Source diff

Build

Tests

step 1 approved and committed: Add formatter architecture

Language: C++

Figure 24. Approving architecture is an explicit phase gate, not an incidental result of approving one proposal. [Back to explanation](#)

8. Implementation workflow

Implementation targets functions that are stubs or have no definition. ICODA builds a deterministic, persisted bottom-up queue: callees are implemented before callers. Recursive call cycles are treated as one component with a stable name/signature/file ordering.

Choose the next target

The queue row shows the current target or batch and the number of remaining callables. To override the head, right- click a pending callable and choose **Implement this function**. The override is persisted and becomes the next two-round implementation target.

Figure 25. A target override changes the next review unit without discarding the deterministic queue.

Scope controls which remaining targets are eligible:

Scope	Meaning
Queue order	Continue through the persisted bottom-up order.
Single entity	Limit the next selection to the current target.
Enclosing class	Bring remaining callables of the current class together.
Enclosing cluster	Bring remaining callables of the current dependency cluster together.
All remaining leaves	Select remaining call-graph leaves before their callers.

Changing scope reorders the relevant queue tail and invalidates an already approved approach.

Batch size is the maximum ordinary queue batch. A later candidate joins only when it has no unimplemented dependency outside the candidate batch. Keep it at 1 for the smallest possible review unit.

Grouping has two modes:

- **One entity** uses the ordinary queue/batch path. With Batch size greater than one, an ordinary batch can still contain multiple entities; use Batch size 1 for a literal one-entity step.
- **Few-line group** deliberately combines an adjacent getter/setter family or overload set only when all members are in one file and enclosing class, each fits `max_function_lines`, and the total fits `hard_max_function_lines`. The queue row shows the exact refusal when grouping is unsafe.

Few-line grouping also requires successful recorded test evidence for every member before approval.

Figure 26. Grouping is visible as a bounded batch and adds an explicit evidence requirement for every member.

Round 1: approve the approach

1. Press **Propose approach**.
2. Review the prose plan, expected entities, and expected files in the **Approach** tab.
3. Press **Approve approach**, **Reject...**, or **Adapt...**

An approved approach is persisted for the current queue batch. Changing the target, scope, batch size, or grouping invalidates it. The approach round does not modify source files.

Figure 27. The first round presents a prose plan, expected entities, and expected files before any code is generated.

Round 2: approve code and tests

1. Press **Propose** after approving the approach.
2. Inspect the delta, structured entity summary, source diff, Build output, and Tests output.
3. Confirm signature changes if present.
4. Approve, reject, adapt, or edit/rebuild the worktree.

On approval, ICODA repeats build and tests in the real project before committing. It advances the queue cursor only after all gates pass and the commit path succeeds. The approved approach is then cleared for the next target.

Figure 28. The second round makes source, tests, model delta, and gate results reviewable together.

Automatic approval

Auto-approve while gates pass defaults off. When enabled, it can approve only a live implementation code proposal whose build and tests pass. It never automatically approves:

- architecture steps;
- implementation approaches;
- historical records;
- failed or missing build/test gates;
- unconfirmed signature changes.

After automatically approving code, ICODA proposes the next approach and stops for the developer's decision. If that approach is approved while auto-approve remains enabled, ICODA starts the code round and can approve the green result. Any refusal is shown in the review panel and status bar.

Figure 29. Automatic approval is deliberately state-aware and reports why it will not proceed.

Filter:

Neighborhood: 0=off

Collapse all

File ViewCall ViewClass ViewMind MapCoverageIssues

Root: main

From mainDepth: 3callers

STATUS — gray stub · blue implemented · green tested · dashed ? uncertain dynamic call

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

Phase implementationRequest (empty: the agent chooses)

Max entities Batch size

Overridden target: describe — 2 remaining

Scope Grouping ☐ Auto-approve while gates pass

Propose approachApprove approach

ProposeApprove architectureApproveConfirm signaturesReject...Adapt...RebuildOpen worktreeUndo last stepCommit manual edits

Step 1: Add configured answer (attempt 1) — batch: centroid, describe

Build: passedTests: failedSignature changes: none

The specification requires it.

ApproachDeltaSignature changesEntity summarySource diffBuildTests

Few-line group test coverage (recorded evidence required for every entity):

- centroid
- describe

sample_project: 13 files, 88 entities, 259 relations, 7 clusters, clustering: seeded_label_propagation | libclang: none found

Language: C++

Figure 25. A target override changes the next review unit without discarding the deterministic queue. [Back to explanation](#)

Version 0.1.0 | Illustrated edition

41

Filter:

Neighborhood: 0=off

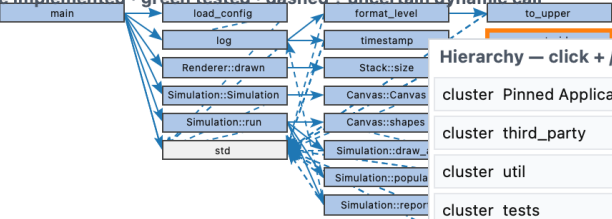
Collapse all

File ViewCall ViewClass ViewMind MapCoverageIssues

Root: main

From mainDepth: ☐ callers

STATUS — gray stub · blue implemented · green tested · dashed ? uncertain dynamic call



Hierarchy — click + / -

- cluster Pinned Application Core
- cluster third_party
- cluster util
- cluster tests
- file config.cppm
- file simulation.cppm

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

Phase implementationRequest (empty: the agent chooses)

Max entities Batch size

Current group (2): centroid, describe — 2 remaining

Scope Grouping ☐ Auto-approve while gates pass

Propose approachApprove approach

ProposeApprove architectureApproveConfirm signaturesReject...Adapt...RebuildOpen worktreeUndo last stepCommit manual edits

Step 1: Add configured answer (attempt 1) — batch: centroid, describe

Build: passed Tests: failed Signature changes: none

The specification requires it.

ApproachDeltaSignature changesEntity summarySource diffBuildTests

Few-line group test coverage (recorded evidence required for every entity):

- centroid
- describe

step 2: signature changes confirmed; approval is now available

Language: C++

Figure 26. Grouping is visible as a bounded batch and adds an explicit evidence requirement for every member. [Back to explanation](#)

Filter:

Neighborhood: 0=off

Collapse all

File ViewCall ViewClass ViewMind MapCoverageIssues

Root: main

From mainDepth: 3callers

STATUS — gray stub · blue implemented · green tested · dashed ? uncertain dynamic call

Hierarchy — click + / -

cluster Pinned Application Core

cluster third_party

cluster util

cluster tests

file config.cppm

file simulation.cppm

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

Phase implementationRequest (empty: the agent chooses)

Current target: centroid — 1 remaining

Scope Grouping ☐ Auto-approve while gates pass

Propose approachApprove approach

ProposeApprove architectureApproveConfirm signaturesReject...Adapt...RebuildOpen worktreeUndo last stepCommit manual edits

Auto-approve stopped — there is no current implementation proposal to approve automatically

Build: not runTests: not runSignature changes: none

ApproachDeltaSignature changesEntity summarySource diffBuildTests

Awaiting developer approval

Use std::ranges::find, keep the existing signature, and add a focused test; about 8 lines.

Expected entities:

.....

sample_project: 13 files, 88 entities, 259 relations, 4 clusters, clustering: seeded_label_propagation | libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib

Language: C++

Figure 27. The first round presents a prose plan, expected entities, and expected files before any code is generated. [Back to explanation](#)

Filter:

Neighborhood: 0=off

Collapse all

File ViewCall ViewClass ViewMind MapCoverageIssues

1 class / structs

STATUS — gray stub · blue implemented · green tested

class service.Formatter
0 data · 1 methods
def normalize(self, value: str) -> str [stub]

Hierarchy — click + / -

cluster .

-

file project.py

file service.py

-

class Formatter

-

method normalize

function main

LLM

Binary

claude

Model

claude-fable-5-1

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

Phase implementationRequest (empty: the agent chooses)

Current target: service.Formatter.normalize — 2 remaining

Scope

Queue order

Grouping

One entity

☐ Auto-approve while gates pass

Propose approach

Approve approach

Propose

Approve architecture

Approve

Confirm signatures

Reject...

Adapt...

Rebuild

Open worktree

Undo last step

Commit manual edits

Step 2: Implement Formatter.normalize (attempt 1)

Build: passedTests: passedSignature changes: none

Keep this atomic and derive the review delta from the changed source.

ApproachDeltaSignature changesEntity summarySource diffBuildTests

1 entities added, 1 changed, 0 removed; files: service.py, tests/test_service.py
+ function tests.test_service.test_normalize def test_normalize(formatter: Formatter) -> None
+ module tests.test_service
...
+ module tests.test_service

step 2: proposal ready — approve, reject or adapt

Language: C++

Figure 28. The second round makes source, tests, model delta, and gate results reviewable together. [Back to explanation](#)

Version 0.1.0 | Illustrated edition

44

Filter:

Neighborhood: 0=off

Collapse all

File View | Call View | Class View | Mind Map | Coverage | Issues

STATUS — gray stub · blue implemented · green tested

Hierarchy — click + / -

cluster Pinned Application Core

cluster third_party

cluster util

cluster tests

file config.cppm

file simulation.cppm

file geometry.cppm

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

Phase implementation

Request (empty: the agent chooses)

Max entities Batch size

Overridden target: describe — 2 remaining

Scope Grouping ☒ Auto-approve while gates pass

Propose approach

Approve approach

Propose

Approve architecture

Approve

Confirm signatures

Reject...

Adapt...

Rebuild

Open worktree

Undo last step

Commit manual edits

Auto-approve stopped — there is no current implementation proposal to approve automatically

Build: not run Tests: not run Signature changes: none

Approach

Delta

Signature changes

Entity summary

Source diff

Build

Tests

No approach approved for the current target.

auto-approve stopped: there is no current implementation proposal to approve automatically

Language: C++

Figure 29. Automatic approval is deliberately state-aware and reports why it will not proceed. [Back to explanation](#)

Version 0.1.0 | Illustrated edition

45

9. Build and test gates

Build and test results are separate. A successful build cannot compensate for failed, absent, or unconfigured tests.

For C++ projects, the default proposal gates are:

```
cmake --preset debug
cmake --build --preset debug
ctest --preset debug
```

For generated projects, `build.sh debug build-only` or `build.cmd debug build-only` provides the separate build-only contract, while running those scripts without `build-only` also invokes CTest for manual use.

For Python projects, the build gate byte-compiles `src/`, and the test gate uses the Code Profile's **Test runner**. Targeted test identifiers are appended when ICODA can associate tests with the selected functions.

Test selection uses explicit model associations and successful step-log provenance. The Tests tab says whether a focused subset or the full suite ran. Test output is retained with the step; callable coverage is credited only when a successful recorded test identifier can reach that callable through the call graph.

Figure 30. A failed test gate remains visible with the exact focused tests; build success cannot make the proposal approvable.

Proposal build and test processes have a 15-minute timeout. Displayed output is limited to the final 6,000 characters so a failing process cannot flood the GUI or step log.

The C++ test command defaults to `ctest --preset debug` in `.icoda/state.json`. There is currently no GUI editor for that field. Change `test_command` only with ICODA closed, preserve it as a JSON array of command arguments, and reopen the project.

Filter:

Neighborhood: 0=off

Collapse all

File ViewCall ViewClass ViewMind MapCoverageIssues

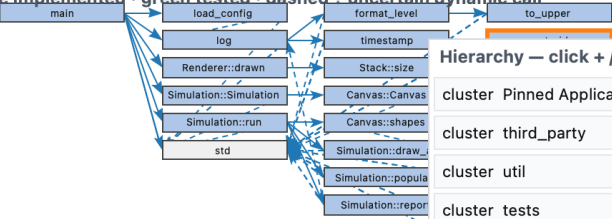
Root: main

From main

Depth:

☐ callers

STATUS — gray stub · blue implemented · green tested · dashed ? uncertain dynamic call



Hierarchy — click + / -

cluster Pinned Application Core

cluster third_party

cluster util

cluster tests

file config.cppm

file simulation.cppm

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

Phase architecture

Request (empty: the agent chooses)

Max entities Batch size

Implementation queue: inactive

Scope Grouping ☐ Auto-approve while gates pass

Propose approach

Approve approach

Propose

Approve architecture

Approve

Confirm signatures

Reject...

Adapt...

Rebuild

Open worktree

Undo last step

Commit manual edits

Step 1: Add configured answer (attempt 1) — the proposal tests fail

Build: passed Tests: failed Signature changes: none

The specification requires it.

Approach

Delta

Signature changes

Entity summary

Source diff

Build

Tests

Selected tests:

- tests/answer_test.cpp

- app::answer_returns_configured_value

step 2: signature changes confirmed; approval is now available

Language: C++

Figure 30. A failed test gate remains visible with the exact focused tests; build success cannot make the proposal approvable. [Back to explanation](#)

10. Git, persistence, and recovery

Project files

Path	Purpose	Normally committed?
<code>.icoda/specification.json</code>	Formal project specification	Yes
<code>.icoda/state.json</code>	Phase, queue, cursor, approach, scope, grouping, and auto-approve state	Yes
<code>.icoda/steps.jsonl</code>	Append-only development and decision history	Yes
<code>.icoda/layout.json</code>	Cluster names and pins	Yes
<code>.icoda/cache/model.json</code>	Derived analysis cache	No
<code>.icoda/ui.json</code>	Per-project provider and UI choices	No
<code>.icoda/icoda.log</code>	Analysis and GUI diagnostic log	No
<code>.icoda/worktree/</code>	Isolated live proposal checkout	No

The generated `.icoda/.gitignore` ignores `cache/` and `ui.json`; step preparation also ignores `worktree/` and `icoda.log`.

Manual edits

ICODA refuses to start a later proposal while ordinary uncommitted project files exist. Choose **Project > Commit Manual Edits** to analyse those edits, record them as a manual step, and commit them. If proposal startup detects a dirty tree, the GUI offers this path directly.

Undo

Undo last step creates a Git revert commit for the most recent approved ICODA step. It works only when the working tree is clean and HEAD is still that step's commit. ICODA will not rewrite unrelated later history; use Git manually when those conditions no longer hold.

Interrupted proposals

A leftover `.icoda/worktree` is preserved and reported when a project is reopened. Inspect it before proposing again if it contains work worth keeping. Starting a fresh proposal resets or recreates the proposal worktree from the current HEAD.

Corrupt state

Invalid JSON in `.icoda/state.json` is refused rather than silently replaced with defaults. The error is shown in the application and logged, and an existing proposal worktree is preserved. Repair or restore the state file from version control before continuing.

User configuration

Global defaults and recent projects are stored at:

- macOS: `~/Library/Application Support/ICODA/config.json`
- Linux: `${XDG_CONFIG_HOME:-~/.config}/icoda/config.json`
- Windows: `%APPDATA%\ICODA\config.json`

The editor value may be a command template containing `{file}` and `{line}`. Without one, ICODA uses VS Code when code is on PATH, then falls back to the operating system's default opener.

11. Troubleshooting

The model is empty or stale

For C++, build the project and confirm a current `compile_commands.json` exists. Open **Project > Choose libclang Library...**, select a detected candidate, press **Apply**, and restart ICODA so the library can be loaded in a fresh analysis process. The chooser distinguishes the saved active choice from the library already loaded by the current process.

If libclang is unavailable or parsing crashes, ICODA keeps the last derived model and marks it stale. Details and the last parsed unit are in `.icoda/icoda.log`.

Figure 31. The chooser makes toolchain discovery explicit and separates the next-start selection from the loaded process.

For Python, syntax-invalid files remain visible with parse errors but contribute no entities or call relations until fixed.

C++ modules do not build

Check all of the following:

- CMake is at least 3.28.
- The selected Clang is module-capable and is accompanied by `clang-scan-deps`.
- CC and CXX point to matching compiler binaries.
- On macOS, Homebrew LLVM is installed or explicitly selected.
- A build cache copied from another checkout has been removed.

Generated build scripts detect a mismatched `CMAKE_HOME_DIRECTORY` and replace that preset's stale cache.

A proposal action is disabled

- In specification, save a valid specification first.
- In architecture, **Propose** is available; approach and queue controls are not.
- **Approve architecture** requires no live proposal.
- In implementation, approve an approach before requesting code.
- **Approve** requires a usable proposal with passing build and tests.
- Signature changes require **Confirm signatures**.
- **Adapt** requires a usable live structured proposal or an unapproved approach.
- Historical records are review-only.

Provider invocation fails

Run provider qualification, confirm the binary is on PATH, authenticate it interactively, and verify the model ID is accepted by the installed CLI version. A typed full path must still have a basename matching a configured provider command. Provider stderr and the final error appear in the review panel.

Build passes but approval is unavailable

Open the Tests tab. Tests must report passed, not not run. Confirm the test runner exists in the environment and the test command is valid. For a few-line group, every member also needs reachable recorded test evidence.

External edits are not reflected

Return focus to ICODA or choose **File > Reload**. ICODA snapshots analysed source files and reloads when the window regains focus after a real source change. A changed body can demote a previously tested entity to implemented until tests establish fresh evidence.

Analysis or GUI failure

Read `.icoda/icoda.log` first. For verification failures, open the newest directory named by `.icoda-test-artifacts/LATEST`, then inspect `summary.txt`, the failing check's `.log`, `environment.json`, and the captured screenshots.

Detected libclang libraries

- ☐ Automatic detection
- ☒ /opt/homebrew/opt/llvm@22/lib/libclang.dylib (homebrew) [active]
- ☐ /opt/homebrew/opt/llvm/lib/libclang.dylib (homebrew)
- ☐ /Library/Developer/CommandLineTools/usr/lib/libclang.dylib (clt)

Active: /opt/homebrew/opt/llvm@22/lib/libclang.dylib

Currently loaded: libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib

Automatically selected the first detected libclang library.

Apply

Close

Figure 31. The chooser makes toolchain discovery explicit and separates the next-start selection from the loaded process. [Back to explanation](#)

12. Maintainer guide

Code map

Area	Main files
Desktop shell and File View	<code>icoda.py</code>
Larger Tk widgets	<code>icoda_gui/</code>
Specification	<code>icoda_core/specification.py</code> , <code>specification.schema.json</code> , <code>icoda_gui/spec_editor.py</code>
Analysis and model	<code>icoda_core/analysis.py</code> , <code>python_analysis.py</code> , <code>model.py</code> , <code>session.py</code>
Views and graph behavior	<code>views.py</code> , <code>class_view.py</code> , <code>mind_map.py</code> , <code>graph_filter.py</code> , <code>expansion.py</code>
Workflow and gates	<code>steps.py</code> , <code>phases.py</code> , <code>implementation_queue.py</code> , <code>grouping.py</code> , <code>auto_approve.py</code>
History and coverage	<code>steplog.py</code> , <code>coverage_index.py</code> , <code>requirement_coverage.py</code> , <code>rules.py</code>
Provider integration	<code>agent.py</code> , <code>providers.json</code> , <code>provider_check.py</code>
Persistence and Git	<code>persistence.py</code> , <code>git.py</code>

Keep non-GUI decisions in `icoda_core` whenever possible. Tk widgets should render core results and dispatch explicit actions rather than reimplementing queue, coverage, rule, or phase logic.

Required verification after every change

Run the complete project gate:

```
./verify.bash
```

On Windows:

```
verify.cmd
```

The verifier runs all checks even after one fails and retains evidence for each:

1. `git diff --check`
2. Ruff
3. mypy
4. Python byte compilation
5. local provider qualification
6. C++ sample configure, build, and CTest
7. pytest with branch coverage, JUnit XML, and an 85% minimum
8. fresh sample-project analysis
9. real-Tk GUI acceptance with validated screenshots

Each run creates `.icoda-test-artifacts/<timestamp>/` with command logs, `summary.json`, `summary.txt`, `environment.json`, provider qualification, JUnit, XML/HTML coverage, the sample ICODA log, GUI state, and PNG captures. `.icoda-test-artifacts/LATEST` names the newest directory.

Do not treat a screenshot's existence as visual verification. The acceptance script checks dimensions, colour variation, visible controls, and graph content; maintainers should also inspect captures affected by a GUI change for clipping, overlap, unreadable labels, incorrect state, and stale output.

Figure 32. The deterministic lifecycle ends only after the queue is empty and recorded evidence covers every callable.

Focused acceptance scripts

Run the GUI suite directly when changing views, controls, interaction, or rendering:

```
.icoda-venv/bin/python tests/gui_acceptance.py \  
  --project tests/sample_project \  
  --output .icoda-test-artifacts/gui-manual
```

Run the deterministic full lifecycle when changing phase, proposal, queue, approval, or persistence behavior. The project directory must be absent or empty:

```
.icoda-venv/bin/python tests/simulation_acceptance.py \  
  --project .icoda-test-artifacts/simulation-project \  
  --output .icoda-test-artifacts/simulation-manual
```

Run the retained large-project benchmark when changing analysis, layout, expansion, or coverage algorithms:

```
.icoda-venv/bin/python tests/performance_acceptance.py \  
  --output .icoda-test-artifacts/performance-manual
```

These GUI scripts require a real display. The standard verifier uses `xvfb-run` automatically on headless Linux when it is available.

Real-provider acceptance

Real-provider acceptance consumes model usage and is intentionally excluded from `verify.bash`. Run it only when a provider, prompt, response, or end-to-end generation contract needs qualification, and always use a new output directory:

```
.icoda-venv/bin/python tests/real_provider_acceptance.py \  
  --provider codex \  
  --model gpt-5.6-sol \  
  --output .icoda-test-artifacts/real-provider-codex-YYYYMMDD-HHMMSS
```

It retains the prompt, provider command, stdout/stderr, parsed response, source diff, build/test results, model facts, step records, Git history, approval, undo, and final clean-tree checks.

Documentation and release evidence

- `docs/GAP_ANALYSIS.md` is the feature-to-evidence inventory.
- `docs/SIMULATION.md` records the complete developer-controlled lifecycle.
- `docs/RELEASE_MATRIX.md` records only platform behavior actually executed.
- `docs/VERIFY_LOG.md` is the detailed chronological verification record.

Do not upgrade a platform or feature status based on inference. Preserve exact commands, outputs, screenshots, and environment metadata that support the claim.

Filter:

Neighborhood: 0=off

Collapse all

File ViewCall ViewClass ViewMind MapCoverageIssues

Specification coverage: no requirements or goals

No specification requirements or goals are available; add them in Project → Specification.

Coverage	ID	Kind	Specification item	Implementing entities
----------	----	------	--------------------	-----------------------

Test coverage: 4/4 callables covered · 0 uncovered

Coverage evidence comes from successful recorded test runs and call reachability.

Coverage	Callable	How covered: tests	Evidence
----------	----------	--------------------	----------

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

Phase implementation

Request (empty: the agent chooses)

Max entities Batch size

Implementation queue: empty — no unimplemented functions

Scope Grouping ☐ Auto-approve while gates pass

Propose approach

Approve approach

Propose

Approve architecture

Approve

Confirm signatures

Reject...

Adapt...

Rebuild

Open worktree

Undo last step

Commit manual edits

No proposal

Build: not run Tests: not run Signature changes: none

ApproachDeltaSignature changesEntity summarySource diffBuildTests

step 3 approved and committed: Implement main

Language: C++

Figure 32. The deterministic lifecycle ends only after the queue is empty and recorded evidence covers every callable. [Back to explanation](#)

13. Current limitations

The current implementation has the following important boundaries:

- The specification editor validates a supplied formal specification but does not yet guide the developer through an AI-assisted discovery and choice process.
- Specification coverage is shown in the Coverage table; the all-diagram coverage colour mode represents recorded test provenance, not specification-tag coverage.
- Most Code Profile and rule-check findings are advisory rather than approval-blocking.
- C++ proposal gates assume the debug CMake build and test presets.
- Only Claude Code and Codex CLI are enabled in the shipped provider registry. Other provider templates are present but intentionally unqualified.
- The release matrix currently qualifies Linux only. Windows and macOS launcher and platform paths exist, but the repository's current release evidence marks them unqualified and unmeasured.
- Retained large-project measurements name non-blocking scaling defects in Python parsing, File View layout, and expansion derivation. Consult `docs/GAP_ANALYSIS.md` before making performance claims.

14. Command reference

```
# Start the GUI
./icoda.bash [project-directory]

# Install contributor dependencies
./icoda-venv/bin/python -m pip install -e '.[dev]'

# Run the complete verification gate
./verify.bash

# Qualify local provider CLIs without model usage
./icoda-venv/bin/python -m icoda_core.provider_check \
  --output .icoda-test-artifacts/provider-qualification.json

# Analyse a project in the same child-process path used by the GUI
./icoda-venv/bin/python -m icoda_core.session /absolute/path/to/project

# Build and test a generated C++ project
./build.sh debug

# Build a generated C++ project without the script's CTest stage
./build.sh debug build-only
```

Windows equivalents use `icoda.cmd`, `verify.cmd`, `.icoda-venv\Scripts\python.exe`, and `build.cmd`.

15. Glossary

Approach: the prose-only first round for an implementation target or batch.

Derived model: ICODA's parsed representation of files, entities, relations, status, documentation, and test associations.

Entity: a parsed namespace, type, function, method, field, variable, alias, or related source construct.

Gate: a condition that must pass before approval, especially build, tests, delta validation, phase consistency, signature confirmation, and grouped test coverage.

Implementation queue: the persisted bottom-up order of remaining stub or undefined callables.

Proposal: a structured provider response applied and verified in the isolated worktree.

Step record: one JSON Lines history entry describing a phase transition, approach decision, approved/rejected proposal, manual edit, or undo.

USR: a stable entity identifier used by the derived model and implementation queue.

Worktree: `.icoda/worktree`, the detached Git checkout in which a live proposal is applied and tested before it can affect the project working tree.